UNIVERSIDAD DE LOS LLANOS — INGENIERÍA DE SISTEMAS

Procesos Estocásticos (Código 603504) | Proyecto Final HMM 2D/3D

AUDITORÍA TÉCNICA Y ARQUITECTURA DEL PROYECTO

Organización modular, responsabilidad por carpeta/archivo y trazabilidad con la especificación

1. ¿Por qué está organizado así el proyecto? (Principios de Diseño)

La estructura del repositorio responde a una **arquitectura modular en capas con Programación Orientada a Objetos (POO)** y **separación estricta de responsabilidades**, exigida por el enunciado oficial (**§4.1 y §4.2**):

1. **Separación entre inferencia en tiempo real y entrenamiento offline (§4.1):** La aplicación gráfica (`main.py` + `gui/`) **nunca entrena** ni captura datasets: únicamente carga al inicio los 54 modelos pre-entrenados desde `models/` (`2d_*.json` y `3d_*.json`). El entrenamiento Baum-Welch, el aumento de datos y la evaluación viven aislados en `training/`.
2. **Aislamiento del motor estocástico puro (§3.1 y §4.2):** `core/HMMEngine.py` implementa de forma **100% nativa sobre NumPy** Forward escalado, Backward, Viterbi en log-space, Baum-Welch, proyección futura y Bayes multiclase, sin depender de PySide6, OpenGL ni librerías externas de ML (`hmmlearn`, `scikit-learn`, `scipy.stats`).
3. **Desacoplamiento geométrico 2D/3D (§3.4 y §4.2):** `core/FeatureExtractor.py` transforma trazos crudos en secuencias discretas de símbolos $O_{1:T}$ y reconstruye trayectorias proyectadas sin conocer cómo funciona el HMM ni cómo se pintan los widgets.
4. **Fuente única de verdad en configuración:** `config/config.toml` centraliza las 27 letras del español, alias ASCII (`ñ → enie`), parámetros de cuantización (`K = 16`, `resample_step = 0.05`, `expected_letter_size = 250.0`, `expected_angle_3d = 1.57`, `sphere_radius = 1.0`) y estados $N_c \in [4, 11]$ por letra.

2. Mapa de Carpetas y Flujo de Datos

```
Proyecto_estocastico-main/
├── config/    → Parámetros globales únicos (alfabeto, discretización, estados Bakis)
├── core/      → Núcleo matemático y geométrico independiente de la GUI
├── data/      → Dataset de trazos reales (2,560 muestras) y plantillas canónicas (27)
├── training/  → Scripts offline de aumentación, entrenamiento y evaluación K=8 vs K=16
├── models/    → 54 modelos HMM serializados en JSON (27 para 2D + 27 para 3D)
├── gui/       → Interfaz gráfica PySide6 + OpenGL (Ventanas A, B, C, Guía y Ranking)
├── docs/      → Enunciado original, SPEC actualizado, auditoría y sustentación
├── tests/     → Suite automatizada Pytest (33 pruebas de unidad e integración)
└── main.py    → Orquestador de arranque de la aplicación de escritorio
```

Flujo Offline: `config/config.toml` + `data/glifo-dataset-2026-11-12.json` → `training/train.py` (usando `core/FeatureExtractor.py` y `core/HMMEngine.py`) → exporta 54 JSON en `models/`.

Flujo en Vivo (GUI): `main.py` carga `models/*.json` en modo solo lectura → eventos de ratón en `View2D`/`View3D` → `FeatureExtractor` → `HMMClassifier` → actualiza `RankingPanel` y proyección punteada verde.

3. Inventario Detallado por Carpeta y Archivo

3.1 Raíz y Configuración (`main.py`, `config/`)

Archivo	Líneas	Clases / Funciones y Responsabilidad
`main.py`	165	`HMMRecognizerApp`, `main()`. Carga los 27 clasificadores 2D y 27 3D desde `models/`, instancia extractores y construye la barra lateral con 4 vistas: *Inicio*, *Modo 2D*, *Modo 3D* y *Guía de letras*.
`requirements.txt`	7	Fija 5 dependencias: `numpy==2.5.4`, `PySide6==6.12.0`, `pyqtgraph==0.14.0`, `PyOpenGL==3.1.10`, `pytest==9.1.1`. Cero librerías externas de HMM/ML.
`pytest.ini`	3	Configura `testpaths = tests` para descubrimiento automático de pruebas.
`config/config.toml`	92	Define `[alphabet]` (27 minúsculas + alias `ñ=enie`), `[features]`, `[capture]`, `[evaluation]` y `[hmm.states]` (`N ∈ [4, 11]`).

3.2 `core/` — Motor Matemático y Extractor Geométrico (Sin GUI)

Archivo	Líneas	Clases / Funciones y Responsabilidad
`core/config.py`	141	`FeatureConfig`, `ProjectConfig`, `load_config()`, `get_config()`. Parsea `config.toml` con `tomllib`, valida 27 letras minúsculas sin dígrafos (`ch`, `ll`) ni tildes.
`core/HMMEngine.py`	581	**Núcleo estocástico puro en NumPy:** `HMM` (`forward` escalado, `backward`, `viterbi` log-space, `fit` Baum-Welch con suavizado circular angular `0.78/0.11/0.11`, `project_future_symbols`, `length_deviation_penalty`, `shape_penalty`, `jump_penalty`, serialización JSON), `init_bakis_hmm()`, `init_segmental_bakis_hmm()` y `HMMClassifier` (`classify` con Log-Sum-Exp y `predict_and_project`).
`core/FeatureExtractor.py`	582	`compute_shape_features()` (vector $g \in R^7$), `FeatureExtractor` (`quantize_direction`, `quantize_jump`, `_moving_average`), `FeatureExtractor2D` (`normalize`, `resample`, `extract`, `slice_prefix`, `reconstruct_projected_strokes`) y `FeatureExtractor3D` (`build_camera_matrix`, `rotate_to_camera_frame`, `cartesian_to_spherical` con `np.unwrap`, `resample_geodesic` con SLERP, `directions_spherical`, `step_geodesic`, `intersect_ray_sphere`).

3.3 `gui/` — Interfaz Gráfica de Usuario (PySide6 + OpenGL)

Archivo	Líneas	Clases / Funciones y Responsabilidad
`gui/MainMenu.py`	198	**Ventana A:** `MainMenu`, `_ModeCard`, `_render_2d_icon()`, `_render_3d_icon()`. Pantalla de inicio minimalista "*Elige un modo*".

`gui/View2D.py`	322	**Ventana B:** `RecognizerCanvas2D` y `View2D`. Captura 2D en tiempo real, usa `realtime=True` mientras se dibuja y `realtime=False` al soltar el trazo (≥35 px), acota la proyección verde en el estado final y ofrece *Reiniciar captura* y *Limpiar lienzo*.
`gui/View3D.py`	545	**Ventana C:** `SphereCanvas3D`, `View3D`, `_slerp_arc()`, `_densify_spherical()`, `_build_guide_frame_curves()`. Esfera OpenGL con **recuadro guía azul de ±48°**, ray-casting exacto, rotación, zoom, *Centrar esfera*, *Girar esfera*, *Reiniciar captura* y *Limpiar lienzo*.
`gui/RankingPanel.py`	259	`RankingPanel`, `_LetterRankRow`, `load_canonical_glyphs()`, `build_handwritten_glyph_pixmap()`. Lista ordenada de las 27 clases con glifo manuscrito y probabilidad $P(c \mid O_{1:t}, g)$.
`gui/LetterGuide.py`	345	`GuideCanvas` y `LetterGuideView`. Sincronizada con `canonical_glyphs.json`, muestra orden de sub-trazos (ej. `d` = `c` + palo vertical), círculos `1` y `2`, flechas, animación paso a paso y práctica con ratón.

3.4 `data/`, `models/`, `training/`, `tests/` y `docs/`

Ruta	Contenido y Responsabilidad
`data/glifo-dataset-2026-11-12.json` `data/canonical_glyphs.json`	Dataset maestro con **2,560 muestras** reales (74 a 162 por letra, incluyendo puntos de 1 clic para `i/j` y variantes de `v/n`) y 27 plantillas canónicas normalizadas en $[0, 1]^2$.
`models/2d_*.json` y `3d_*.json`	54 archivos JSON con los parámetros aprendidos de los 27 modelos 2D y 27 modelos 3D listos para inferencia instantánea.
`training/augment.py` (209 lín.) `training/train.py` (161 lín.) `training/evaluate.py` (181 lín.)	Pipeline offline: carga de dataset preservando trazos de 1 punto, proyección 3D sobre S^2, aumento afín, entrenamiento unificado Baum-Welch (`_train_letter`, `_train_all`) y evaluación de prefijos `30%, 50%, 70%, 100%` comparando `K=8` vs `K=16`.
`tests/` (4 archivos, 33 tests)	`test_hmm_engine.py` (8), `test_feature_extractor.py` (12), `test_training_pipeline.py` (3) y `test_gui.py` (4, verificando ausencia de entrenamiento en GUI, navegación de 4 vistas, proyección 2D y ray-casting 3D).
`docs/`	`enunciado-proyecto-final-reconocedor-hmm.pdf`, `SPEC.pdf`, `AUDITORIA_ARQUITECTURA.pdf`, `SUSTENTACION_MATEMATICA.pdf` y `ecuaciones.md`.